

Doubly Linked List – Insert At Position And Time Complexity.

Program :

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next = newNode;
    if (newNode->next != nullptr)
```

```

    {
        newNode->next->prev = newNode;
    }
    cout << data << " inserted at position " << pos << "."
<< endl;
}

... .

    case 3:
        cout << "Enter data to insert: ";
        cin >> data;
        cout << "Enter position: ";
        cin >> pos;
        insertAtPosition(data, pos);
        break;

```

Program Analysis:

```

if (pos < 1)
{
    cout << "Invalid position!" << endl;
    return;
}

```

If pos or input position by user less than 1 ,then it will give output ``Invalid position`` and return void and exit from function.

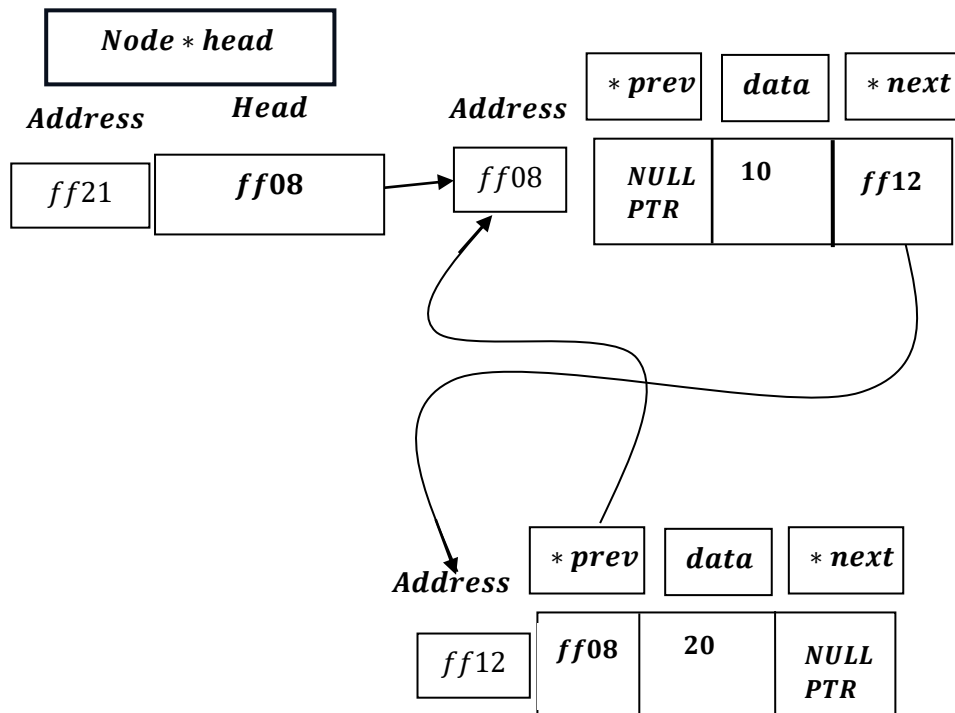
```

if (pos == 1)
{
    insertAtBeginning(data);
    return;
}

```

If pos or input position by user equal to 1 ,then it run func: *inserAtBeginning* , passing integer user input ``data`` i.e. *insertAtBengining(data)* to insert a node with data at beginning .

Now ,say we have a list:

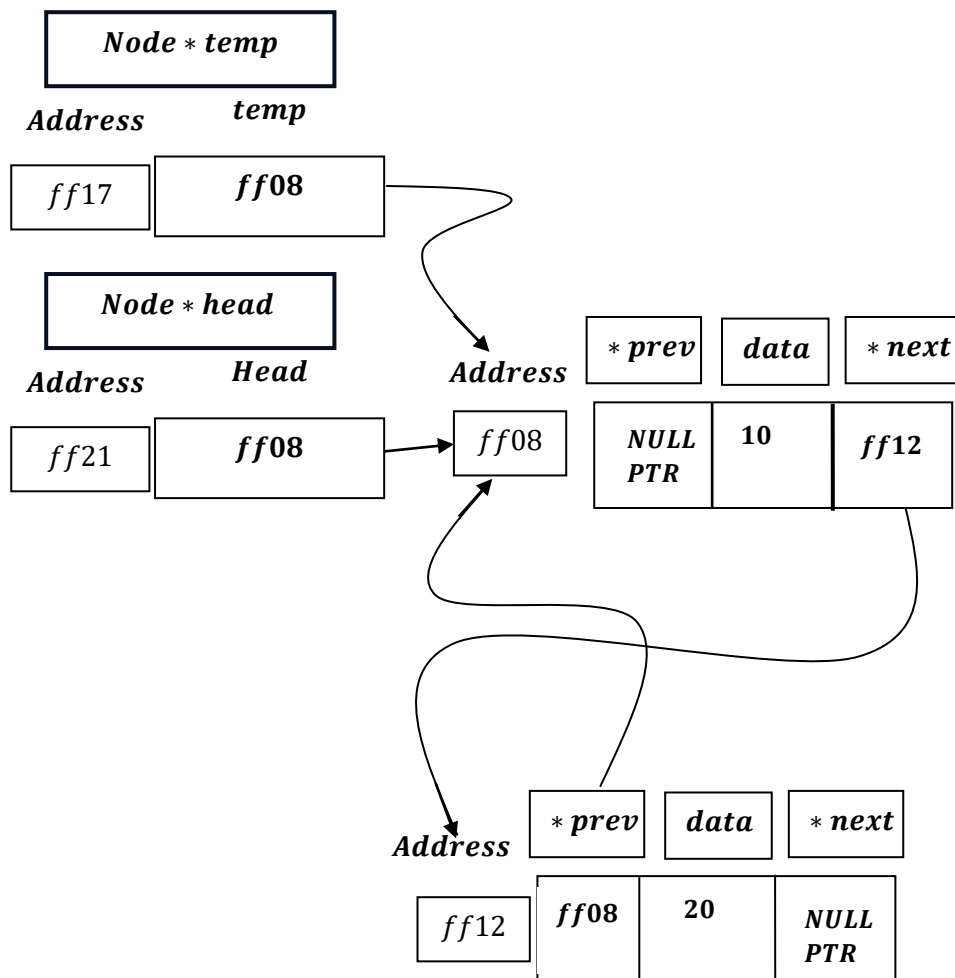


Now, we want to insert data 15 at pos: 2.

Next,

```
Node *temp = head;
```

```
Node * temp = head = ff08;
```



Next,

```
for (int i = 1; i < pos - 1 && temp != nullptr; i++)
{
    temp = temp->next;
}
```

So, pos is: 2 and $i = 1$ and $i < 2 - 1$ i.e. $i < 1$, which implies $i = 1$ and $i < 1$ is contradictory and condition of for loop fails though $temp \neq NULLPTR$ is True. Hence, as condition fails, it exits from for loop.

```
if (temp == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

temp = ff08 and is not nullptr, hence, condition check fails or becomes false.

Next,

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

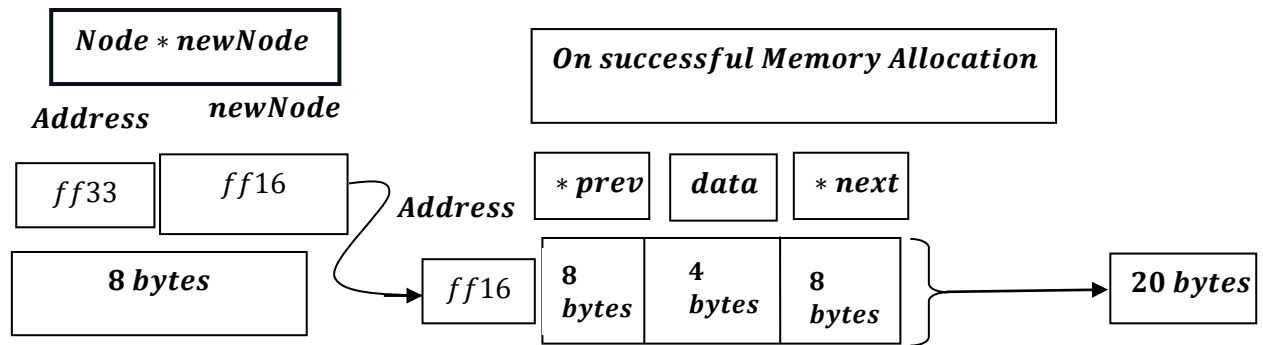
→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

- data(an integer) is typically 4 bytes.*
- next(a pointer to another Node) is typically 8 bytes.*
- prev(a pointer to previous Node) is typically 8 bytes.*

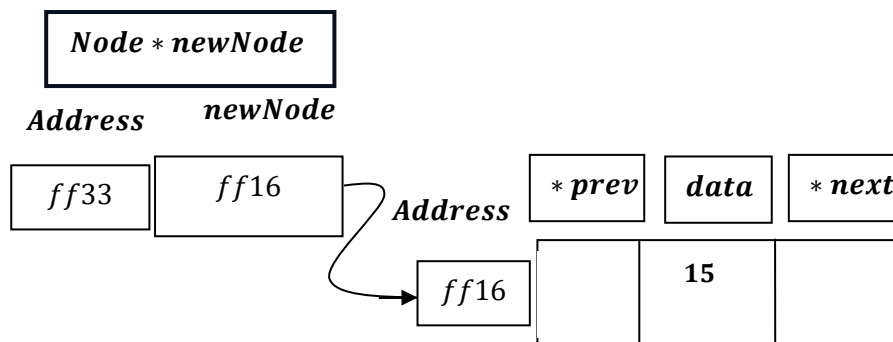
→ Therefore, the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for previous) .

Say,



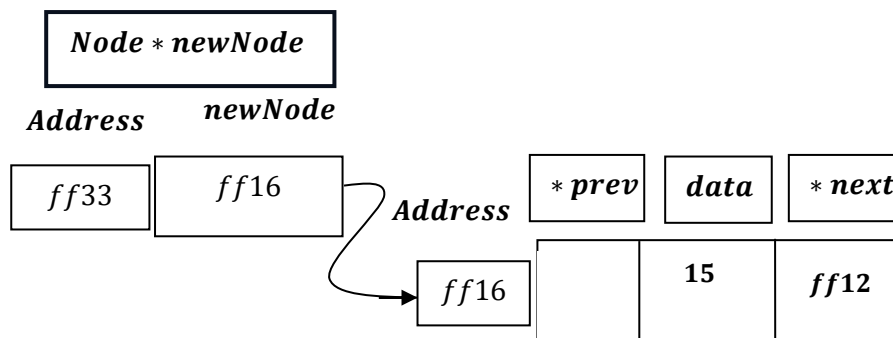
```
newNode->data = data;
```

newNode → *data* = *data* = 15;

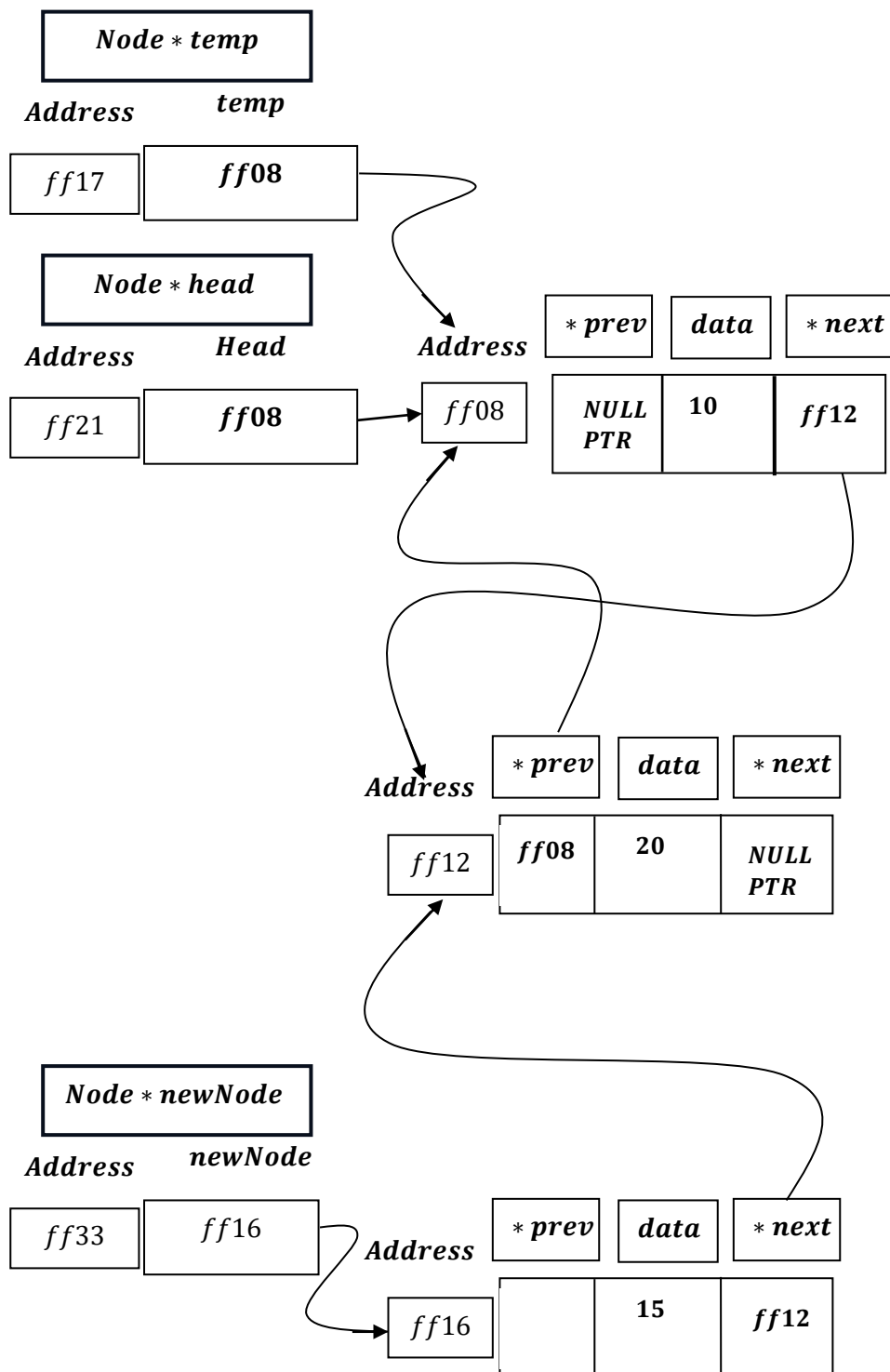


```
newNode->next = temp->next;
```

newNode → *next* = *temp* → *next* = ff12.

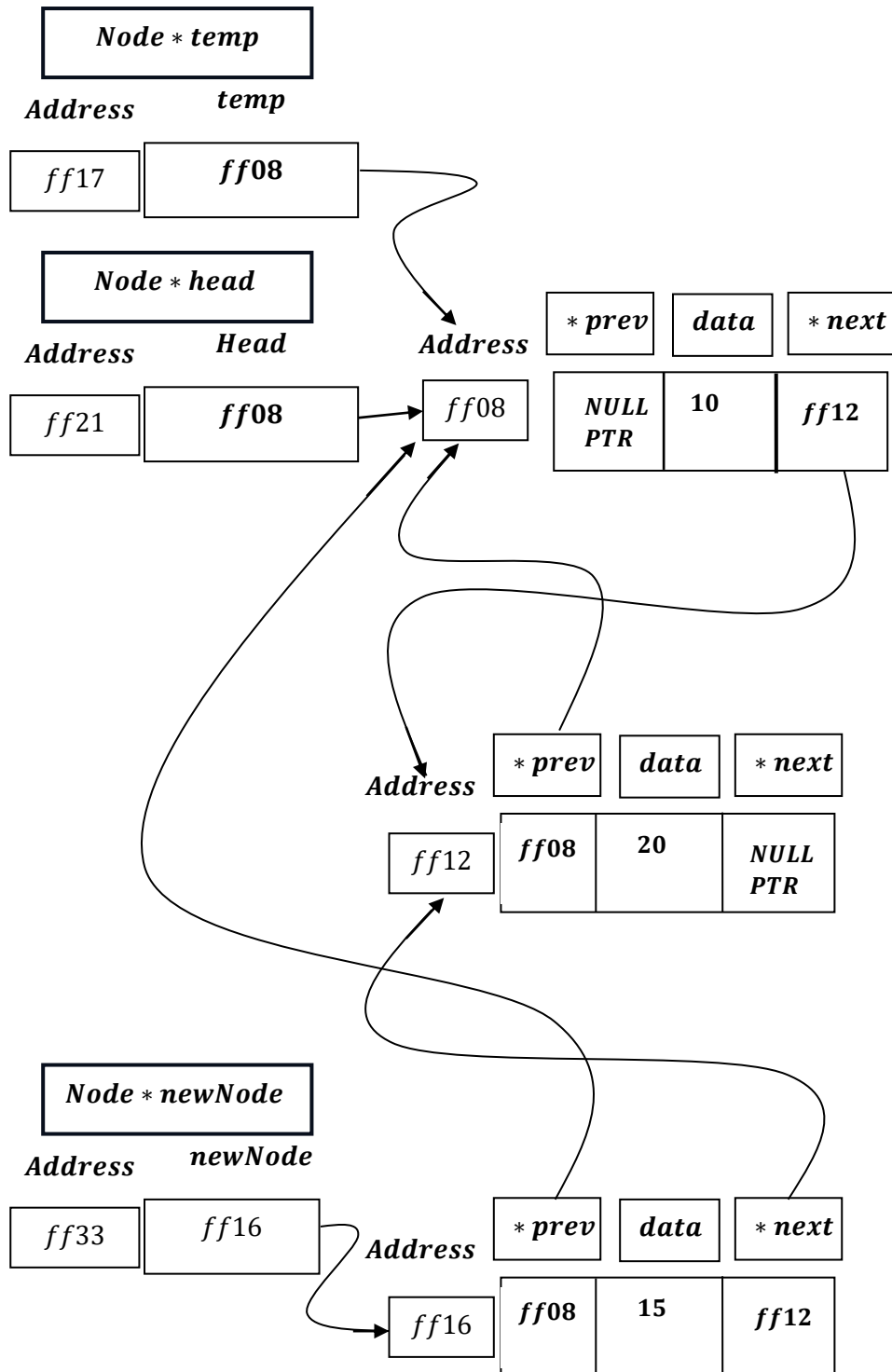


And now, next pointer will point to ff12 address.



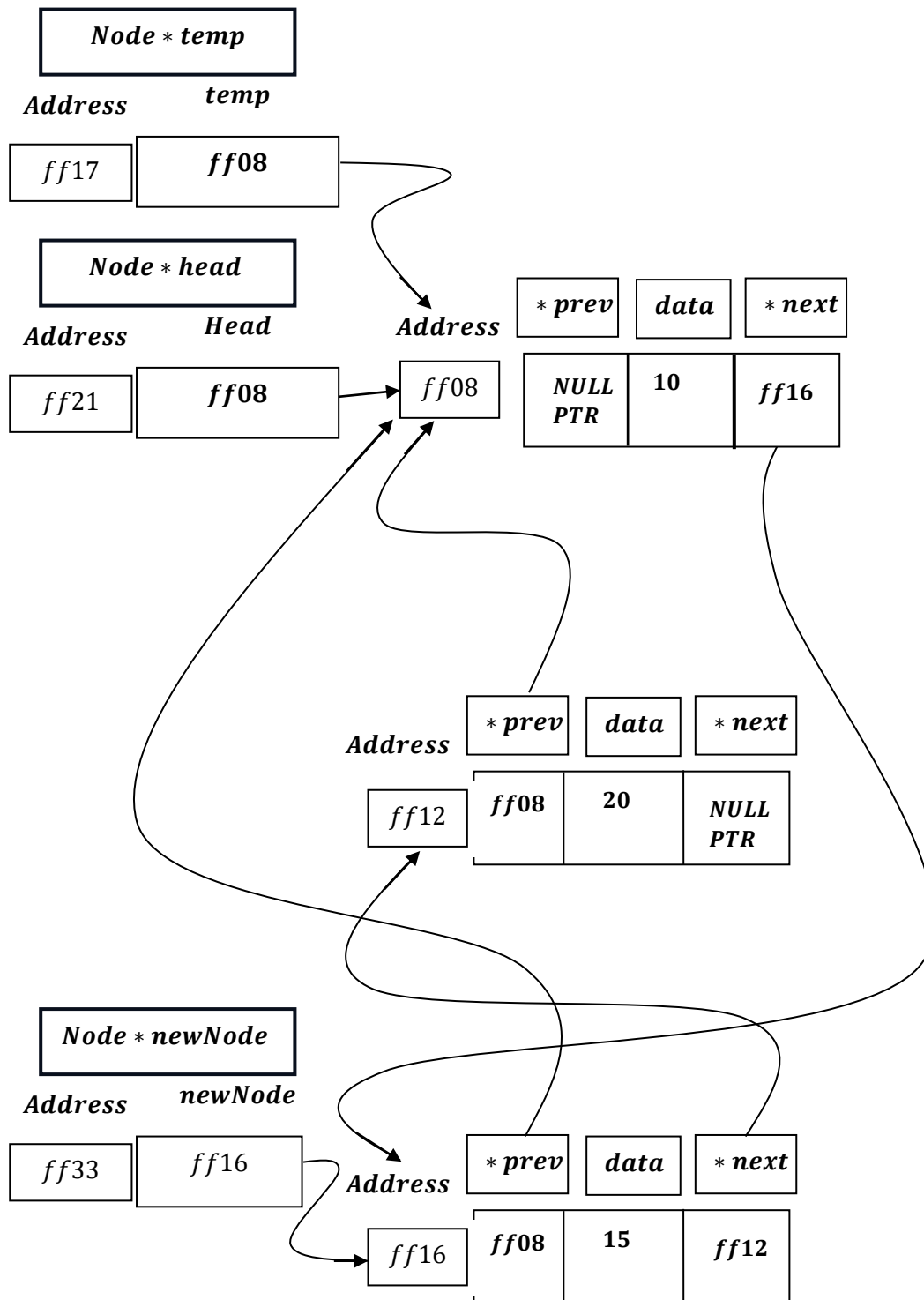
```
newNode->prev = temp;
```

newNode → *prev* = *temp* = *ff08*



```
temp->next = newNode;
```

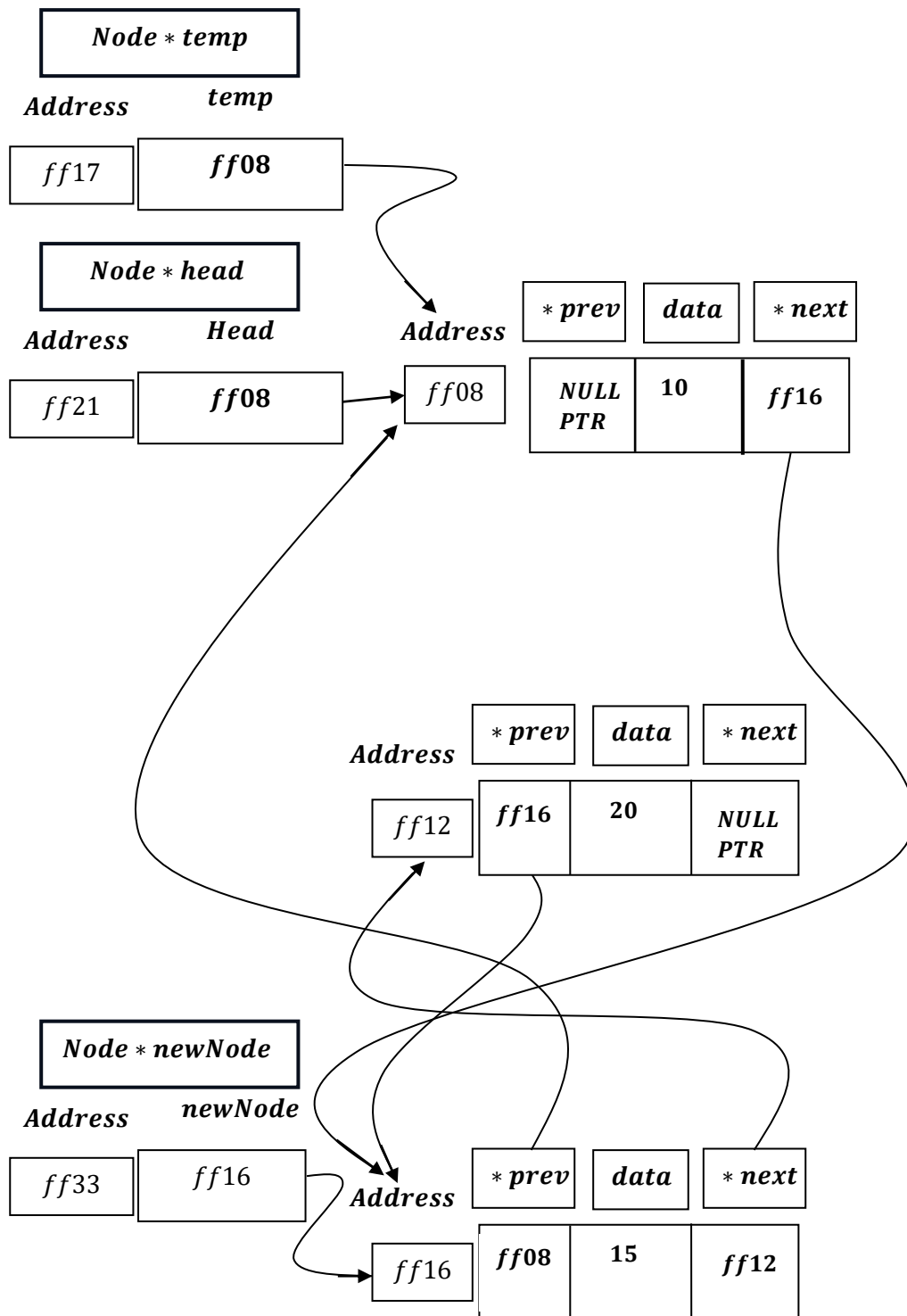

$temp \rightarrow next = newNode = ff16$



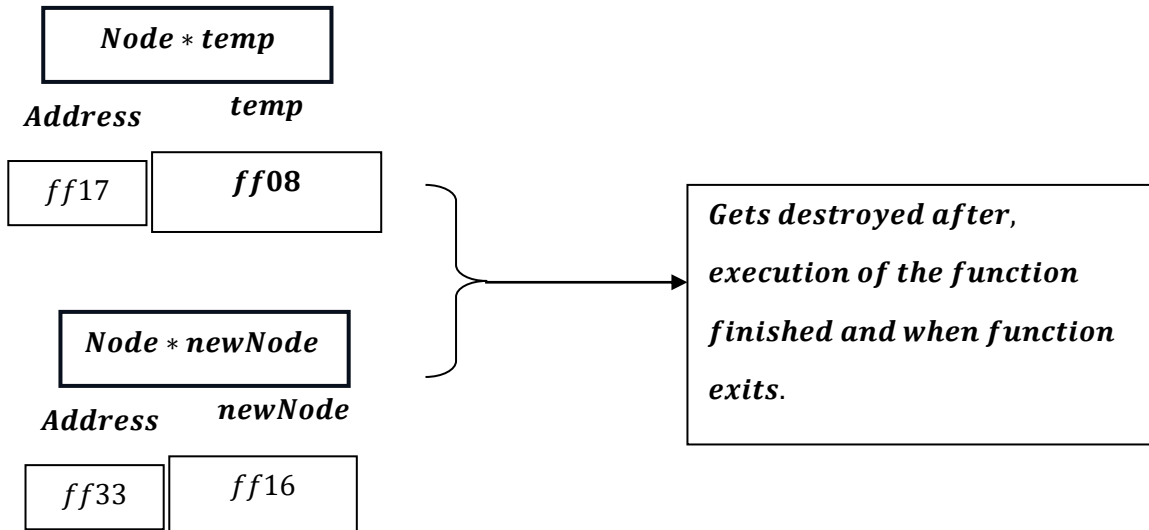
```
if (newNode->next != nullptr)
{
    newNode->next->prev = newNode;
}
```

if $newNode \rightarrow next \neq nullptr$, as $newNode \rightarrow next = ff12$, then:

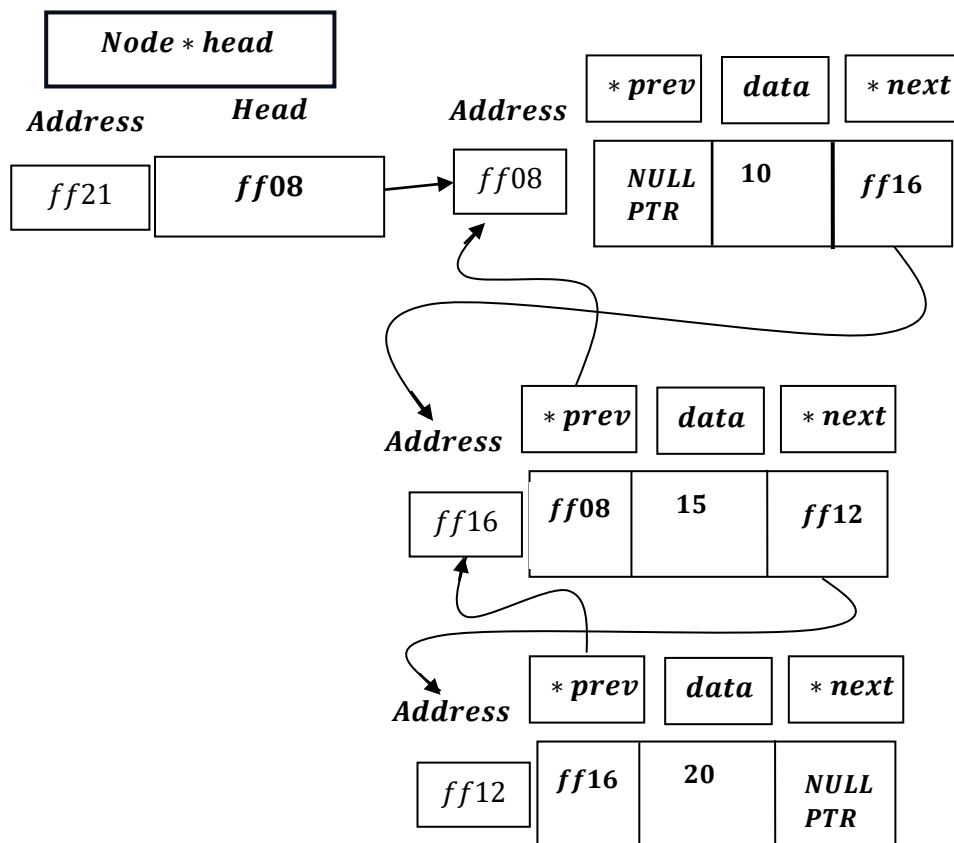
- *$newNode \rightarrow next \rightarrow prev = newNode = ff16$.*



Now, *Node * newNode* , *Node * temp* temporary variables, therefore gets destroyed after function execution get finished .

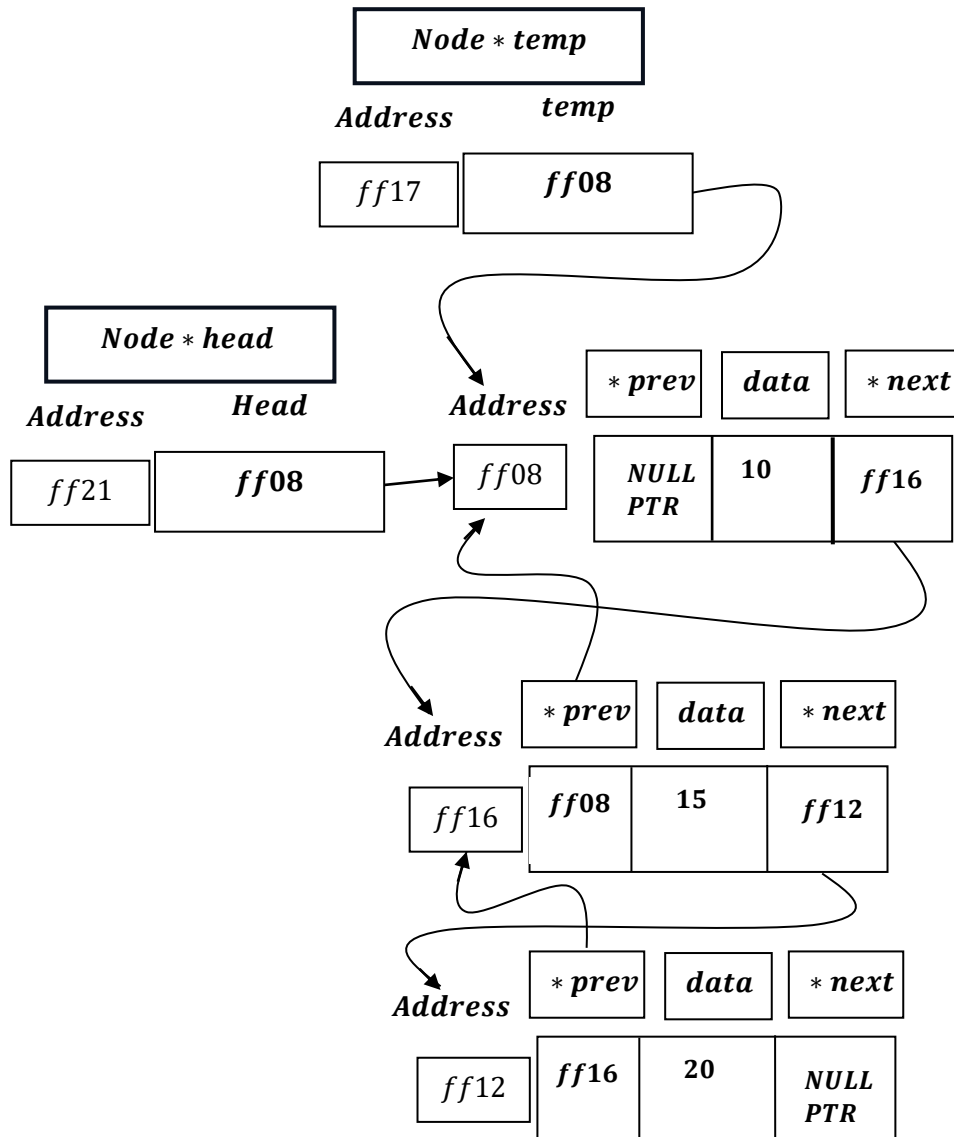


Hence ,we are left with :



When temp = nullptr?

Suppose, we have list:

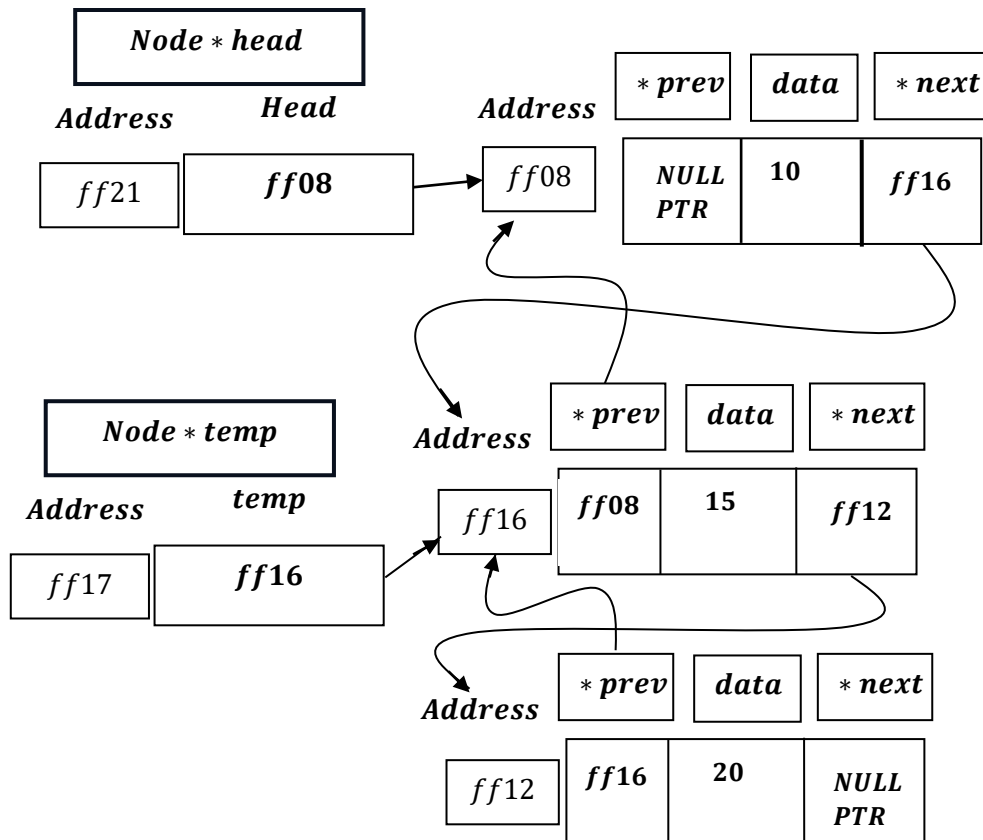


And ,pos input by user 5 ,as input 4 will be the last node, thats why condition is :
 $i = 1$ and $i < pos - 1$. As When we will insert the last $(n + 1^{th})$ node ,
temp pointer will traverse to $\rightarrow n^{th}$ node .

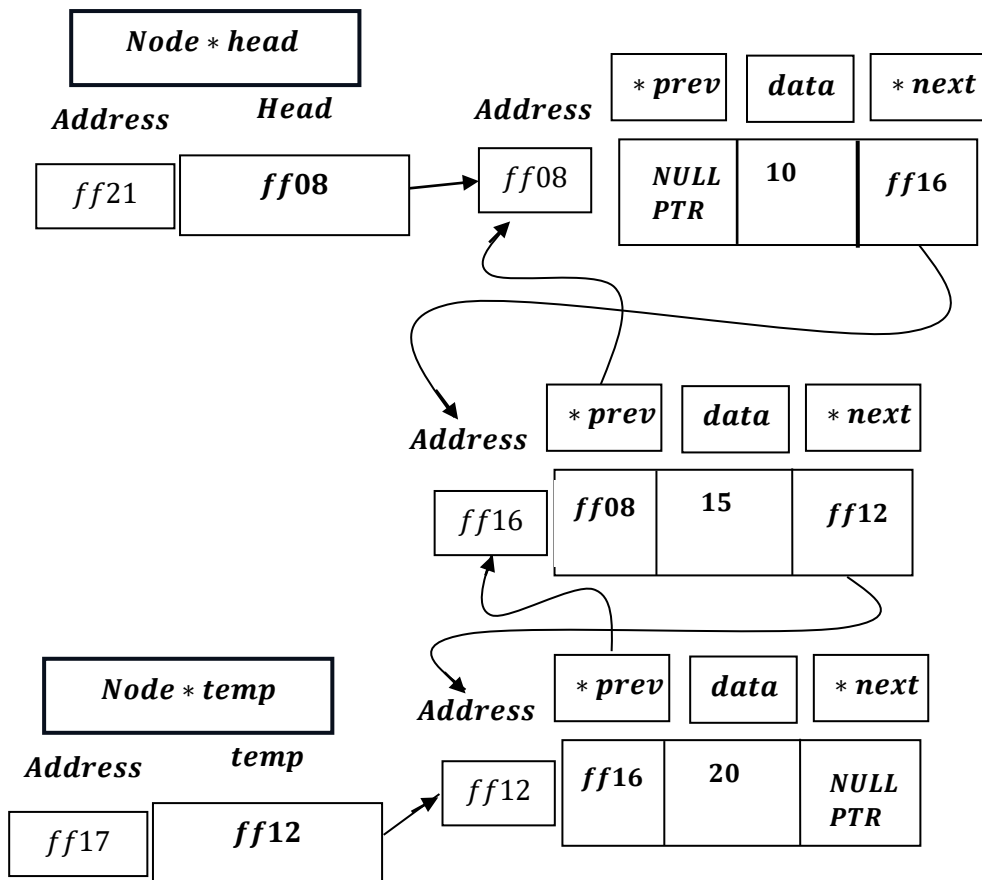
As ,pos input 4 ,`i` will be first 1 , i.e. first traversal and then

$i < (4 - 1) = i < 3$ i.e. $i = 2$ i.e. second traversal , implies traversal needed 2 times to point to n^{th} node.

$pos = 4, i = 1$

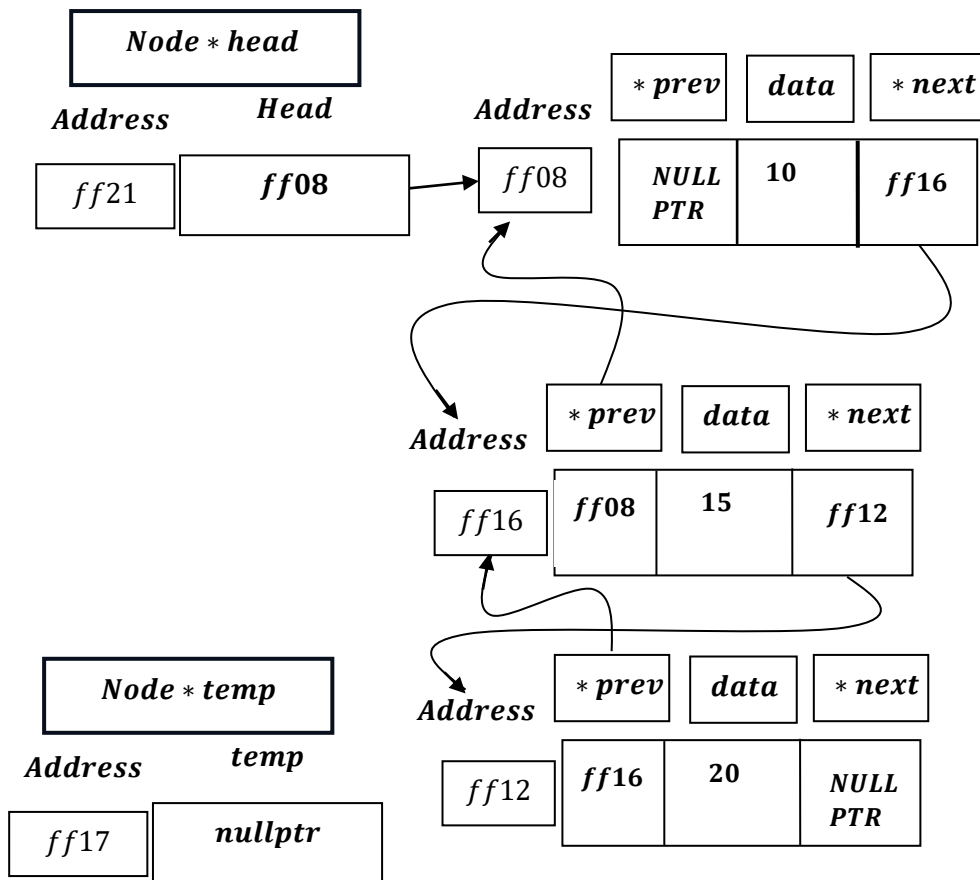


$pos = 4, i = 2[i < (4 - 1 = 3)]$



But if $n = 5$, then:

$$pos = 5, i = 3[i < (5 - 1 = 4)]$$



As, there is no such node hence temp pointer will be assigned to ``nullptr``.

```
for (int i = 1; i < pos - 1 && temp != nullptr; i++)
{
    temp = temp->next;
}
```

For condition fails as temp = nullptr and loop exits.

```
if (temp == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```


Now, temp = NULLPTR, hence, it gives output : "Position out of bounds!"

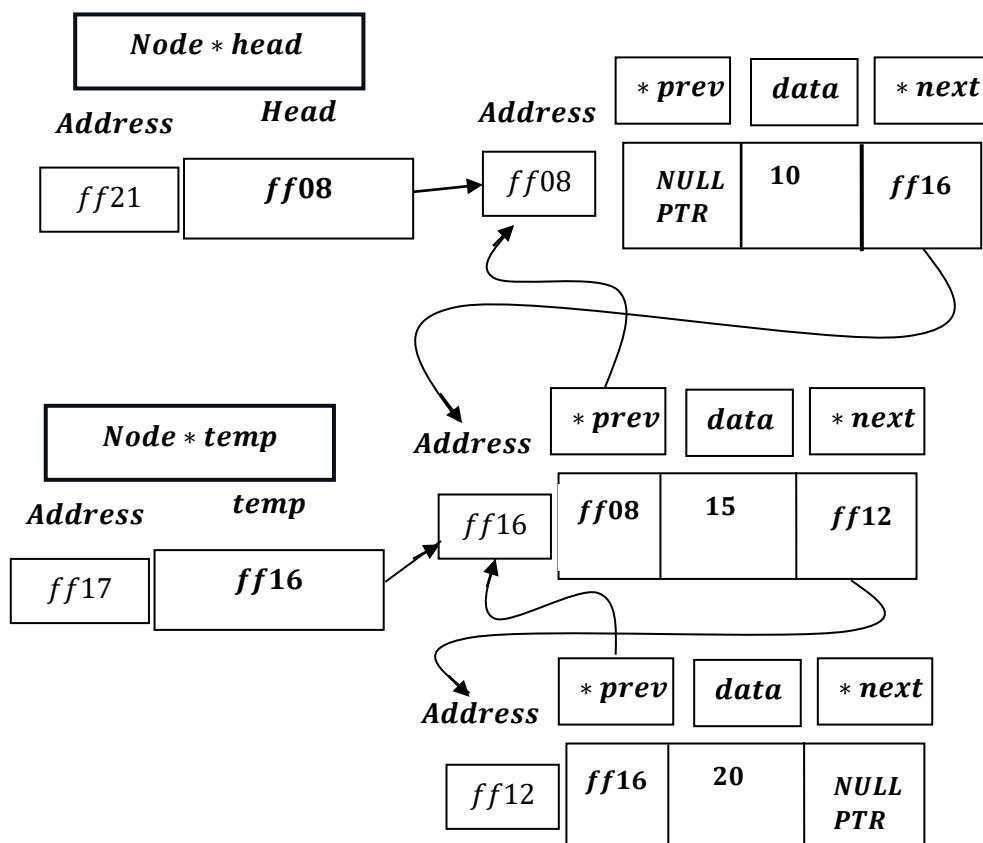
And exits from function.

When newNode → next will be NULLPTR according to function here?

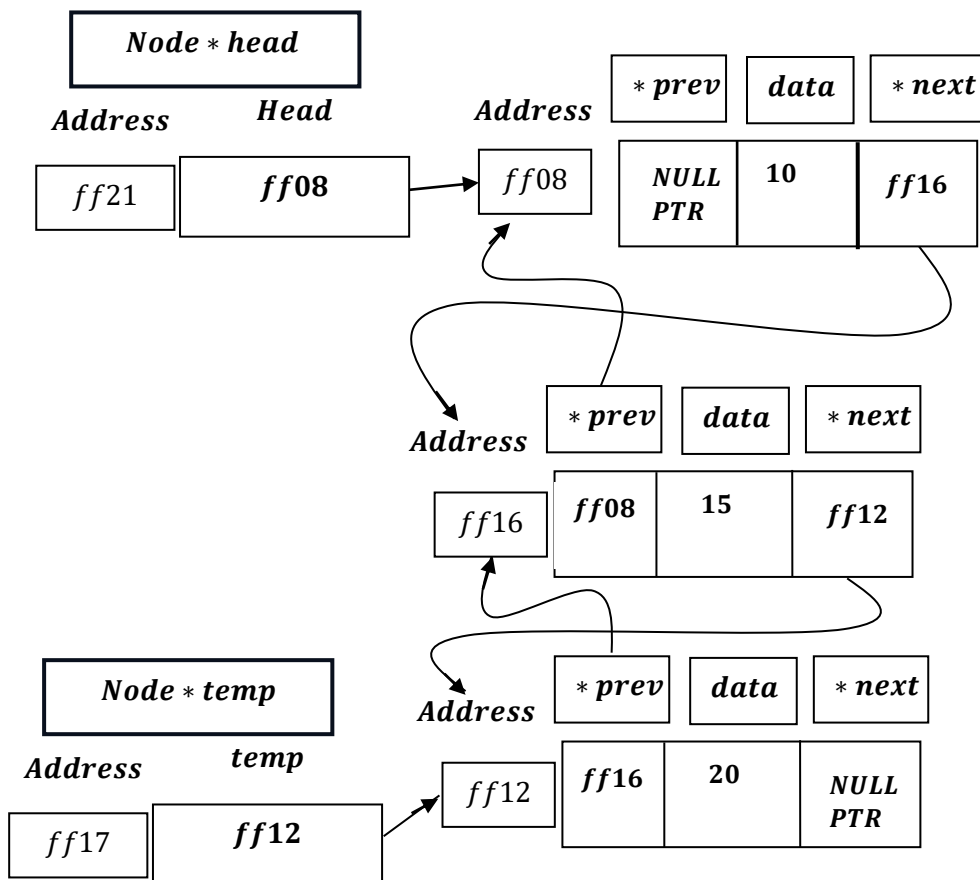
Answer is just simple when we are inserting node at the end and

temp → next = NULLPTR.

pos = 4, i = 1



$pos = 4, i = 2[i < (4 - 1 = 3)]$



And for loop exists ,

```
newNode->next = temp->next;
```

$newNode \rightarrow next = temp \rightarrow next = NULLPTR.$

Hence,

```
if (newNode->next != nullptr)
{
    newNode->next->prev = newNode;
}
```

If condition is false here and statement will not run when we try to insert node at end due to `newNode → next → prev` doesn't exist and `newNode → next = NULLPTR`.

Doubly Linked List - Insert At Position [Working Summary]:

This function inserts a new node at a specific numerical position in the list.

→ **First, it handles edge cases:**

- If the position (`pos`) is less than 1, it's considered invalid, and the function exits.
- If the position is 1, it calls the `insertAtBeginning` function to handle the insertion and then exits.

→ **For any other position, it traverses the list.** A temporary pointer, `temp`, starts at the `head` and moves forward `pos - 2` times. The goal is to make `temp` point to the node at position `pos - 1` (the node that will be *just before* the new node).

→ **After the traversal, it performs a check.** If `temp` is `nullptr`, it means the requested position was out of bounds (the list is shorter than the position), so it prints an error and exits.

→ **Finally, it performs the insertion.** A new node is created. The function then re-wires four pointers to fit the new node between the node at `pos - 1` (`temp`) and the node at the original position `pos` (`temp->next`).

1. The **new node's** `next` points to the original node at `pos`.
 2. The **new node's** `prev` points back to the node at `pos - 1` (`temp`).
 3. The **next pointer of the node at `pos - 1` (`temp`)** is updated to point to the new node.
 4. The **prev pointer of the original node at `pos`** is updated to point back to the new node (this step is skipped if inserting at the very end of the list).
-

Time Complexity of Insert At Position Function Module

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1) → O(1)
    {
        cout << "Invalid position!" << endl; → O(1)
        return; → O(1)
    }
    if (pos == 1) → O(1)
    {
        insertAtBeginning(data); → O(1)
        return; → O(1)
    }

    Node *temp = head; → O(1)
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }
    → 1 + 1 + ... to pos - 2 times = O(pos - 2) or, O(pos) ≈ O(n).
    → runs 'pos - 2' times at worst condition.

    if (temp == nullptr) → O(1)
    {
        cout << "Position out of bounds!" << endl; → O(1)
        return; → O(1)
    }

    Node *newNode = (Node *)malloc(sizeof(Node)); → O(1)
    newNode->data = data; → O(1)

    newNode->next = temp->next; → O(1)

    newNode->prev = temp; → O(1)

    temp->next = newNode; → O(1)
}
```

```

if (newNode->next != nullptr) → O(1)
{
    newNode->next->prev = newNode; → O(1)
}
cout << data << " inserted at position " << pos << "."
<< endl; → O(1)
}

```

There are three case scenarios:

- 1. *Position is less than 1 or 1 and 2 , i.e. we want to insert data at first or second . (Best Case)*
- 2. *Position is $N + 1$, i.e. we want to insert data position at end . (Worst Case)*
- 3. *Inserted position from 3 to $N + 1$, for loop runs from 1 to $N - 1$. While if position is 1 and 2 , it takes constant amount of time : $O(1)$. And when the position is uniformly distributed from 1 to $(N + 1)$. (Average Case i.e. Undetermined Position). That is we donot whether the position will be 1 or 2 or $N + 1$, where Node is to be inserted.*

1. When Position is less than 1 or 1 and 2 (Best Case)

1. A. When Position is less than 1 or equal to 1 [Absolute Best Case]:

1. *If position `pos` less than 1, it outputs ``Invalid Position`` and return void , which exits from the function, and these process takes constant amount of time , typically $O(1)$.*

2. *if position `pos` is equal to 1 , then it runs insertAtBeginning function , we know this function takes constant amount of time , typically $O(1)$ with retun void statement ,*

(return statement) which makes function to exit, process takes constant amount of time $O(1)$.

1. B. When Position is equal to 2 [Best Case but not the Absolute Best Case]

1. If position `pos` less than 1, which is false, this check takes constant time, $O(1)$.

2.. if position `pos` is equal to 1, which is false, this check takes constant time, $O(1)$.

3. Temp pointer of Node named Structure, its value assigned to head pointer's value i.e. the address to which head pointer points to, making Temp pointer points to that address. This operation takes constant amount of time i.e. $O(1)$.

4. Now for loop, $i = 1$ and $i < (pos - 1)$ i.e. $i < (2 - 1)$ i.e., $i < 1$, the condition contradicts itself, hence for loop does not run, the condition checking takes constant amount of time i.e. $O(1)$.

5. newNode pointer of Node named structure points and allocates a memory for the Node structure, allocating for prev pointer, integer data, and next pointer, this pointing as well as allocating through `malloc` takes constant amount of time i.e., $O(1)$.

6. integer data in the memory allocated pointed by newNode pointer of Node structure set to data by user input, takes constant amount of time i.e., $O(1)$.

7. next pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by next pointer of allocated memory pointed by temp pointer of Node structure and it makes next pointer pointed by newNode to point to that address, which is pointed by next pointer pointed by temp pointer, the process takes constant amount of time i.e., $O(1)$.

8. *prev pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address)obtained by temp pointer of Node structure and it makes prev pointer pointed by newNode pointer point to that address ,which is pointed by temp pointer ,the process takes constant amount of time i.e., $O(1)$.*

9. *next pointer of allocated memory pointed by temp pointer of Node structure set to the value(address) obtained by newNode pointer of Node structure and it makes next pointer pointed by temp pointer point to that address ,which is pointed by newNode pointer ,the process takes constant amount of time i.e., $O(1)$.*

10. *Now ,If condition,checks next pointer pointed by newNode pointer of Node structure not equal to `nullptr` ,which is true then prev pointer pointed by next pointer pointed by newNode pointer of Node structure set to the value i.e.address obtained by newNode pointer ,making prev pointer pointed to address ,which is being pointed by newNode. ,the process takes constant amount of time i.e., $O(1)$.*

11. *Now we output a statement,user input data inserted at positon ,the process takes constant amount of time i.e., $O(1)$.*

Hence ,total time complexity it takes : $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$,i.e.constant time complexity.

Why not Absolute Best Case ?

⇒ When pos = 1, loop runs 0 times. [maintaining ,pos – 2 times]

⇒ When pos = 2, loop runs 0 times. [maintaining ,pos – 2 times]

⇒ When pos = 3, loop runs 1 times. [maintaining ,pos – 2 times]

.

.

.

⇒ When n = n, loop runs `pos – 2` times. [maintaining ,pos – 2 times]

⇒ When n = n + 1, loop runs `pos – 2` times. [maintaining ,pos – 2 times]

Loop should run `pos – 2` times as if we want to enter the last node

in the list, i.e., $n + 1$ node then, temp pointer must point to the current last node i.e. $(n)^{th}$ node.

Why $pos - 2$ times is needed because temp must point to Head node or, the node pointed by head at first, then traversal take place, as it already point a node, therefore traversal will be $pos - 2$ times.

If we notice minutely then we will observe that, the loop runs 1 to $n - 1$ times in worst case scenario, being n is the size of the list.

Such as if n is 3 then if we want to insert at $n + 1$ node i.e. $pos = 4$, then, it will take loop to run 2 times i.e. $i < (n + 1) - 1$ or, $i < n$, hence $n - 1$ times.

Therefore when, $pos = 2$, as function remains at constant time, we can take this as best case but not the 'Absolute Best Case'. The 'Absolute Best Case' is when List is Empty.

3. When Position is $N + 1$, Worst Case Scenario

1. If position pos less than 1, which is false, this check takes constant time, $O(1)$.

2.. if position pos is equal to 1, which is false, this check takes constant time, $O(1)$.

3. Temp pointer of Node named Structure, its value assigned to head pointer's value i.e. the address to which head pointer points to, making Temp pointer points to that address. This operation takes constant amount of time i.e. $O(1)$.

4. Now for loop, $i = 1$ and $i < (pos - 1)$ i.e. $i < (pos - 1)$ i.e., $i = pos - 2$, the condition assert itself as true, then:

$temp = temp \rightarrow next$, runs $pos - 2$ times till temp points to current n^{th} node. i.e. statement runs: $1 + 1 + 1 + 1 + \dots (pos - 2) \text{ times} = O(pos - 2) = O(pos)$ and if we substitute pos with n , statement runs: $O(n)$ times.

Alternatively ,

*if n size = 1 , inserting ' $n + 1$ ' node make loop runs 0 times i.e. $i < (n + 1) - 1$ or,
 $i < n$ i.e. $n - 1$ times .*

*if n size = 2 , inserting ' $n + 1$ ' node make loop runs 1 times i.e. $i < (n + 1) - 1$ or,
 $i < n$ i.e. $n - 1$ times .*

*if n size = 3 , inserting ' $n + 1$ ' node make loop runs 2 times i.e. $i < (n + 1) - 1$ or,
 $i < n$ i.e. $n - 1$ times .*

.
.

*if n size = n , inserting ' $n + 1$ ' node make loop runs i.e. $i < (n + 1) - 1$ or,
 $i < n$ i.e. $n - 1$ times .*

*Alternatively, ' $temp = temp \rightarrow next$ ', runs ' $n - 1$ ' times till temp points to
current ' n^{th} ' node i.e. statement runs : $1 + 1 + \dots (n - 1)$ times = $O(n - 1) = O(n)$.*

*5. newNode pointer of Node named structure points and allocates a memory
for the Node structure , allocating for prev pointer , integer data, and next pointer
, this pointing as well as allocating through ' $malloc$ ' takes constant amount of
time i.e., $O(1)$.*

*6. integer data in the memory allocated pointed by newNode pointer of Node structure
set to data by user input , takes constant amount of time i.e., $O(1)$.*

*7. next pointer of allocated memory pointed by newNode pointer of Node structure
set to the value(address) obtained by next pointer of allocated memory pointed by
temp pointer of Node structure and it makes next pointer pointed by newNode to
point to that address , which is pointed by next pointer pointed by temp pointer ,*

the process takes constant amount of time i.e., $O(1)$.

8. prev pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by temp pointer of Node structure and it makes prev pointer pointed by newNode pointer point to that address ,which is pointed by temp pointer ,the process takes constant amount of time i.e., $O(1)$.

9. next pointer of allocated memory pointed by temp pointer of Node structure set to the value(address) obtained by newNode pointer of Node structure and it makes next pointer pointed by temp pointer point to that address ,which is pointed by newNode pointer ,the process takes constant amount of time i.e., $O(1)$.

10. Now ,If condition, checks next pointer pointed by newNode pointer of Node structure not equal to `nullptr` ,which is true then prev pointer pointed by next pointer pointed by newNode pointer of Node structure set to the value i.e. address obtained by newNode pointer ,making prev pointer pointed to address ,which is being pointed by newNode. ,the process takes constant amount of time i.e., $O(1)$.

11. Now we output a statement, user input data inserted at position ,the process takes constant amount of time i.e., $O(1)$.

Hence ,total time complexity it takes : $O(1) + O(1) + O(1) + O(n) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(n)$, i.e. constant time complexity.

**3. Inserted position from 1 to $N + 1$, while loop runs from 1 to N .
And when the position is uniformly distributed from 1 to $(N + 1)$:
(Average Case i. e. Undetermined Position)**

→ If the position is 1, the loop's inner statement runs 0 time.
→ If the position is 2, the loop's inner statement runs 0 time.
→ If the position is 3, the loop's inner statement runs 1 times.
→ If the position is 4, the loop's inner statement runs 2 times.
.
.
→ If the position is $(N + 1)$, the loop's inner statement runs : $N - 1$ times ,
As when $i = N < N$ becomes false , hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

Now, as we take all the possibilites then , we take the best case also i. e. $O(1)$, when position = 1 and 2. Considering `T(P)` is time complexity of position `P` .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ 1 & , \text{when } P = 2. \\ P - 2 \text{ or } N - 1 & , \text{when } P = 3, 4, \dots, N + 1. \end{cases}$$

Now, If there is `N` no. of nodes there we can insert data at $N + 1$ positions.

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.
.

For position $N+1$, the probability of insertion is $\frac{1}{N+1}$.

Hence probability of insertion will always remain $\frac{1}{N+1}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 2 + \dots + \frac{1}{N+1} \times N - 1 \right) + \frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 1$$

$$= \frac{1}{N+1} \times \{(1 + 2 + 3 + \dots + N - 1) + 1 + 1\}$$

$$= \frac{1}{N+1} \times \left\{ \sum_{i=1}^{N-1} i + 2 \right\}$$

$$= \frac{1}{N+1} \times \left(\frac{N(N-1)}{2} + 2 \right)$$

$$= \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1}$$

$$= \frac{N^2 - N}{2N+2} + \frac{2}{N+1}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

Now, taking out the dominant term of numerator and denominator:

- ***In the numerator, $N^2 - N$, the term N^2 grows much faster than N .***

So, for large N , the dominant term is N^2 .

- ***In the denominator, $2N + 2$, the term $2N$ grows much faster than the constant 2. The dominant term is $2N$.***

$$= \Theta\left(\frac{N^2}{2N}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + 2 \times \Theta\left(\frac{1}{N}\right)$$

Now removing the constant `2` we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

$\rightarrow \Theta(N)$ grows linearly with N .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ decreases to a constant, i.e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant} \right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note, by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Approach]:
(Simplest approach)

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.

.

For position $N+1$, the probability of insertion is $\frac{1}{N+1}$.

**Hence probability of insertion will always remain $\frac{1}{N+1}$,
for position 1 to $N+1$.**

Average case complexity : Probability of insertion $\times$ $\left(\begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N+1} \times \left(\sum_{i=1}^{N+1} i \right)$$

$$= \frac{1}{N+1} \times \left(\frac{1}{2} (N+1)((N+1)+1) \right) \left[\text{As, } \sum_{k=1}^n k = \frac{1}{2} (n)(n+1) \right]$$

$$= \frac{1}{N+1} \times \left(\frac{(N+1)(N+2)}{2} \right)$$

$$= \frac{N+2}{2}$$

$$= \frac{N}{2} + \frac{2}{2}$$

$$= \frac{N}{2} + 1$$

Applying Big Theta Θ we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta(1)$$

$$= \frac{1}{2} \times \Theta(N)$$

$$= \Theta(N)$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N}{2} + 1\right)$ or $\Theta(N)$.

Now,

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N}{2} + 1 \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>InsertAt Position</i>	<i>1. Best Case</i>	<i>$\Omega(1)$</i>
	<i>2. Worst Case</i>	<i>$O(n)$</i>
	<i>3. Average Case</i>	<i>$\Theta(n)$</i>
